Evaluation Guide 04: Waypoint Dispatch scored as an enterprise application

An honest score out of 100 for every part of the system:

- the frontend, backend and database;
- the Datathon modelling and the AI layer;
- security, testing and operations;
- above all, problem solving and edge-case handling.

It is judged against the Tech-Triathlon 2026 Challenge Booklet, the code on `main` at `d086270`, and a check of whether cheaper or proven alternatives exist. Sources are listed at the end.

Written 2026-10-16. Companions:

- AI and System Guide 01
- Spring Boot Guide 02
- Testing Guide 03

Contents

1. Overall score
2. Scorecard
3. What is genuinely strong
4. Wrong or expensive decisions, and the cheaper or better way
5. Decisions that look wrong but were right
6. Problem solving and real-world gaps
7. The path from 76 to about 88
8. How the scores were set
9. Sources

---

1. Overall score

76 out of 100.

- **As a 15-day competition build:** this is near the top. The planning engine, the edge-case discipline and the architecture are well above what such a build usually has.
- **As an enterprise product a retail group would run tomorrow:** the engineering is above average, but operations and real-world readiness are not there yet.

View	Verdict
Competition (booklet weights)	Strong on the most weighted criteria: the planning and allocation engine, engineering quality, functional completeness
Enterprise buyer	Not yet safe to depend on: CI off, backups on the same disk, alerts reaching nobody, adoption untested in the field

2. Scorecard

Area	Score	Verdict
Problem understanding and domain fit	**90**	Every booklet problem has a built answer: one queue, recorded deferrals, live progress, proof of delivery, flagged shortfalls, forecasts, offline work
Planning engine (the core problem)	**92**	Chosen by evidence. Over 62 days it matches the exact solver's proven optimum on all 34 days the solver could prove. S1 serves 74 of 85 orders and passes the official validator
Backend architecture	**86**	A modular monolith with boundaries checked by tests, one door for writes, an outbox and version guards. Right shape; heavier than needed in places
Database and data integrity	**88**	Row-level security on every operational table, a role per module, no superuser, exact `numeric` for capacity, UUIDv7, 141 tables
Edge cases and resilience	**82**	A register of 388 cases, 342 with a named test; the system degrades visibly. But one documented behaviour (the offline retry backoff) is never called
Security design	**84**	Deny by default, a re-check inside the transaction, opaque sessions, Argon2id, two CSRF locks, audited denials
Security in operation	**55**	The admin password is in Git history, backups live on the same disk, alerts go nowhere, no dependency scanning
Datathon modelling	**80**	Leakage found and reported honestly (the realistic AUC is 0.957, not 0.977); forecast error about 4%. But no model runs on the deployed system yet
AI integration (LLM, Jev, speech, MCP)	**68**	Excellent guardrails, low value for the effort. Mostly off, in shadow, or blocked on a data agreement
Frontend and UX	**76**	Four installable, offline-capable role apps, three languages, a browser suite for every role. But the Sinhala and Tamil are not reviewed, voice is untested on real phones, and browsers limit background GPS
Testing	**80**	About 3,000 checks across every level. But CI is off, there is no real end-to-end test, and nothing checks the frontend's copies of the backend types
Production operations	**50**	CI blocked by billing, no off-server backup, no restore drill, one VPS, the alert webhook unset
Cost efficiency and right-sizing	**62**	Cheap to run (about USD 58 a month) but expensive to build and maintain: enterprise machinery for 60 vehicles and 4 roles
Real-world adoption readiness	**58**	Little done on training, phone and WhatsApp habits, an SMS fallback, data costs or the real warehouse API

3. What is genuinely strong

1. **Decisions backed by evidence.**
   - **The planning engine:** benchmarked over 62 days, against an exact MIP (CP-SAT, SCIP, HiGHS), OR-Tools routing and ALNS, before it was chosen.
   - **The ML leakage check:** exposed its own optimism. The road-conditions index inflated the validation score, and the realistic production figure was reported instead.
   - Few teams do either.
2. **Edge cases are a system, not a list.**
   - Nine handling patterns cover 388 cases.
   - Each case names its trigger, the required behaviour, a production signal and a test.
   - The rule "a case without a test is a case that is not handled" is the right one.
3. **Security at the right layer.**
   - Scope is enforced in PostgreSQL itself, so a Java bug cannot leak another store's orders.
   - Permission is checked again inside the transaction.
   - An unknown command is a `403`, never a `404`.
   - The pool runs as a role that owns nothing.
4. **Honest degradation.** Every external dependency has a fallback that says so on screen:
   - the model service, falling back to the built-in estimate;
   - the LLM, to "rules only";
   - speech, to the phone's own voice;
   - the warehouse, to order totals;
   - the map tiles, to a plain background.
5. **Booklet compliance.**
   - Present:
     - the README, the seeded accounts and the numbered judge walkthrough;
     - Docker Compose and `.env.example`;
     - the architecture and data-model documents;
     - the AI tool disclosure.
   - The scored Datathon pipeline uses no proprietary API. The booklet forbids that ("Proprietary API-based modelling/preprocessing is prohibited").

4. Wrong or expensive decisions, and the cheaper or better way

#	Decision	Problem	Cheaper or better
1	**A custom, IAM-style permission engine:** 146 actions, versioned JSON policies	For 6 roles this is the heaviest part of identity, and it is security code nobody else maintains or audits	Roles plus scope enforced by RLS cover what the booklet needs. If policy as data is a real goal, **Cedar** is open source, has a Java package and a formally verified engine
2	**A hand-built outbox, relay and module verification**	Spring Modulith already offers an event publication log in the business transaction, retries, and `ApplicationModules.verify()` built on ArchUnit	Spring Modulith would remove much of the platform code. Ours adds multi-replica `SKIP LOCKED` claiming and a database role per module, so this is a partial reinvention rather than pure waste
3	**SERIALIZABLE on every transaction, as well as** `row_version`	Two locks on one door. Serializable adds dependency tracking and aborted transactions to retry; the usual advice is to use it only where an invariant needs it	Read Committed plus the version guard for most commands; Serializable only for allocation and capacity
4	**An LLM to reword explanations**	The rules already write the explanation. The LLM adds data leaving the country, a gateway, caps, masking and a faithfulness check, all for nicer phrasing	Better wording templates in code. Keep the gateway for a feature that truly needs language
5	**Jev, an alpha-stage vendor, in shadow**	Effort spent before the basic baseline exists	Build the labelled set and a TF-IDF plus logistic regression baseline (about 50 lines) first. Try Jev only if the baseline loses
6	**Gemini for voice transcription**	About 70 s per note; raw audio leaves the country; production waits for a data agreement, and the PDPA commencement dates are still being set by Gazette	Let the dispatcher play the audio. If transcription earns its place, a self-hosted Whisper fine-tuned on our own notes. Zero-shot Whisper is about 34% word error rate on Sinhala; domain fine-tunes reach about 22%
7	**In-app chat, voice and transcripts**	Sri Lankan operations run on phone calls and WhatsApp. A separate chat must beat that habit, and it has no SMS fallback when mobile data is off	WhatsApp Business templates in Sinhala and Tamil for store notices, with SMS as the fallback. Keep the in-app thread as the dispatcher's record

#	Decision	Problem	Cheaper or better
8	**GPS every 5 seconds from a web app**	Browsers may stop `watchPosition` when the screen is off, and keeping the screen on drains the driver's own phone	Record at stop events, plus coarse fixes on the road. For true tracking later, a thin native wrapper (for example Capacitor with a background service)
9	**Backups as `pg_dump` files on the same disk**	One failed disk loses the database and its backups together	pgBackRest or WAL-G with continuous WAL archiving to the R2 bucket already in use: point-in-time recovery for about USD 1 a month
10	**A hand-written service worker and queue**	A bug like the never-called backoff is easy to miss	Workbox Background Sync covers queue and replay, though it has its own replay pitfalls. Ours adds lanes and the human-in-the-loop conflict review, so keep it, but fix and test the retry path
11	**16 modules, 141 tables and 153 migrations in about two weeks**	The shape is right, but it is wide. MCP, OAuth, the demo control room, voice and risk review pulled effort away from operations: CI, backups and alerts	Hold breadth until the operational basics are done
12	**A demo business clock inside production code**	Moving "now" on a production system is a risk, even though it is opt-in and audited	Leave it out of the production build, or guard it by environment as well as by a setting

5. Decisions that look wrong but were right

Decision	Why it was right
A modular monolith, not microservices	One team, one VPS, and business decisions that must commit together. Microservices would add sagas, a broker and many deployments for no gain at this size
Our own allocation engine, not OR-Tools in production	Shown by benchmark to be as good or better under these exact constraints. OR-Tools was kept as a test-time benchmark
Plain SQL with JdbcTemplate instead of JPA	Row-level security needs a `SET LOCAL` per transaction, and the version guard must be visible in SQL. An ORM would hide both
Opaque server-side sessions instead of JWTs	Disabling an account or changing a policy takes effect on the next request, not at token expiry
Offline tiers by role	The driver works fully offline, the loader and store are resilient, and the dispatcher is online only. Two dispatchers planning offline would produce plans that cannot be reconciled
Datathon models gated out of production	The data is licensed for the competition only. A config guard stops them answering in production until we retrain on our own deliveries

6. Problem solving and real-world gaps

6.1 Solved well

- **Deferrals:** recorded with a reason, and repeated skips are prioritised.
- **The offline driver:** conflicts held for a person to decide, never merged automatically.
- **The store handover:** proven by a PIN, with only a hash kept.
- **The loader:** shortfalls flagged before the vehicle leaves.
- **The watchers:**
  - an attention watch that rings the dispatcher;
  - automatic questions to a store;
  - an expected arrival time for each store.
- **The peak day:** demand above capacity is allocated with an explained policy, checked by the official validator.

6.2 Missing for the real operation

#	Gap	Effect today
1	**A vehicle breaks down mid-route:** vehicle interchange is not built	dispatchers fall back to phone calls
2	**Real geography:** district-level travel times, no road network or live traffic, approximate outlet GPS	arrival times are estimates, not routed
3	**Real integrations:** the warehouse API is simulated; no ERP or invoicing link; no returns or reverse logistics	the system is an island until these exist
4	**People:** no training material, paper fallback runbook or support line built, only planned	driver adoption in Kandy is the biggest risk, and the build optimises the software rather than the change
5	**Devices and data:** low-end Android, mobile data cost, and a lost or wiped shared tablet are mostly unaddressed, apart from ending its sessions	field failures that tests cannot show
6	**Fuel:** the quota is enforced in planning, but there is no fuel-actuals loop (the fuel log was dropped)	quota use is planned, never measured
7	**Datathon models in use:** none is registered on the deployed system, and production forbids them until retrained on own data	the "intelligence" today is the deterministic fallback plus the rules

7. The path from 76 to about 88

All of these are cheap: most are configuration or about a day of work each.

#	Step	Moves
1	Restore CI; add Dependabot or OWASP dependency-check	security in operation, testing
2	Rotate the admin password, and keep it out of the README from now on	security in operation
3	pgBackRest or WAL-G to R2, plus one timed restore drill	production operations
4	Set `ALERT_WEBHOOK_URL`; add security and AI alerts	production operations
5	One real end-to-end Playwright run against `docker compose up`, over the seeded day from order to receipt	testing
6	Generate the frontend's types from the backend contracts	testing, frontend
7	Fix and test the offline retry backoff	edge cases
8	Register the forecast model on preview, so something learned shows on screen	Datathon modelling
9	A real-phone pilot for voice, GPS and the Sinhala and Tamil wording, with a native reviewer	frontend, adoption
10	Freeze new AI features (LLM, Jev, transcription) until the pilot shows the problem they solve	cost, AI integration

8. How the scores were set

- **The judging weights.** The booklet weights the planning and allocation engine (20%), engineering quality and architecture (25%) and functional completeness (20%) most for the Hackathon. For the Datathon it weights model and architecture (25%), data wrangling and label construction (20%) and performance (20%). Those areas are scored with the same strictness.
- **An enterprise buyer's view.** Production readiness, operations and adoption are scored as such a buyer would: by what would happen on the first bad day.
- **Evidence.** Every number is a judgement from the code, the tests, the documents and the sources below, not a computed metric.

9. Sources

- Tech-Triathlon 2026 Challenge Booklet (`Challenge Booklet.pdf` in the repository root)
- Spring Modulith events reference: https://docs.spring.io/spring-modulith/reference/2.0/events.html
- Spring Modulith 1.0 (InfoQ): https://www.infoq.com/news/2023/08/spring-modulith-1-0
- How Cedar was designed (AWS Security Blog): https://aws.amazon.com/blogs/security/how-we-designed-cedar-to-be-intuitive-to-use-fast-and-safe/
- Cedar open-sourced (InfoQ): https://www.infoq.com/news/2023/06/aws-cedar-open-source/
- OPA vs Cedar (Styra): https://www.styra.com/?p=44382
- Serializable Snapshot Isolation in PostgreSQL (paper): https://arxiv.org/pdf/1208.4179
- PostgreSQL mailing list on serialization failures: https://www.postgresql.org/message-id/1368210905.99645.YahooMailNeo%40web162904.mail.bf1.yahoo.com
- pgBackRest vs Barman vs WAL-G compared: https://www.kunalganglani.com/blog/postgresql-backup-tools-compared.md
- Postgres backups with pg_dump, pgBackRest and WAL-G: https://sumguy.com/postgres-backups-pgdump-pgbackrest-walg/
- Workbox issue #2386 on replay: https://github.com/GoogleChrome/workbox/issues/2386
- PWA geolocation capabilities (Progressier): https://progressier.com/pwa-capabilities/geolocation
- Ionic forum, PWA watchPosition pauses: https://forum.ionicframework.com/t/pwa-geolocation-watchposition-pauses/231595
- Sinhala speech pipeline using Whisper (arXiv 2510.04750): https://arxiv.org/pdf/2510.04750
- UCSC thesis, fine-tuned Sinhala ASR: https://dl.ucsc.cmb.ac.lk/jspui/handle/123456789/5012
- whisper-sinhala-small model card: https://huggingface.co/hlasith/whisper-sinhala-small
- PDPA (Amendment) Bill 2025, Gazette: https://documents.gov.lk/view/bills/2025/3/575-2025_E.pdf
- Personal Data Protection Act (Sri Lanka), Wikipedia: https://en.wikipedia.org/wiki/Personal_Data_Protection_Act_(Sri_Lanka)
- Lex Mundi data privacy guide, Sri Lanka: https://www.lexmundi.com/guides/data-privacy-guide/jurisdictions/asia-pacific/sri-lanka/
- WhatsApp Business API in Sri Lanka (vendor guide): https://chatdaddy.tech/blog/whatsapp-business-api-sri-lanka
- WhatsApp for logistics (LINK Mobility): https://www.linkmobility.com/blog/how-whatsapp-business-improves-logistics-and-transportation-communication
- World Bank, social media for business in Sri Lanka: https://thedocs.worldbank.org/en/doc/2f814a0237cf9df413e70fd0036dda12-0310012021/related/Updated-SM-for-Business-SL-Harini.pdf